Internal Assessment 2:
Security Analysis using Snyk

Group Members:

16010123012 - Aaryan Sharma

16010123017 - Aditey Kshirsagar

16010123023 - Aditya Baheti

Target Application: `thermo_aq` (MERN Stack)

Code Repository: https://github.com/Aaryan-Sharma-5/ThermoAQ

Date: March 31, 2026

---

1. Abstract

As modern software development increasingly relies on open-source dependencies, securing the software supply chain has become a critical challenge. This report documents the practical implementation and evaluation of Snyk, a comprehensive developer-security platform. The objective of this experiment was to perform Static Application Security Testing (SAST) and Software Composition Analysis (SCA) on a local MERN stack application (thermo_aq). By utilizing the Snyk Command Line Interface (CLI) and Web Dashboard, the application's source code and third-party packages were scanned to identify vulnerabilities, assess their severity, and review automated remediation strategies.

2. Objectives

- To understand the principles of "Shift-Left" security and DevSecOps.
- To install and configure the Snyk CLI for local development environments.
- To perform Software Composition Analysis (SCA) to identify vulnerable open-source dependencies within a Node.js/MERN architecture.
- To perform Static Application Security Testing (SAST) to detect security flaws in custom-written source code.
- To analyze vulnerability reports and evaluate Snyk's remediation recommendations.

3. Introduction

In traditional software development lifecycles, security testing is often relegated to the final stages before deployment. Snyk is designed to disrupt this model by enabling "Shift-Left" security, empowering developers to find and fix vulnerabilities as early as the coding phase.

Snyk operates primarily on two fronts: finding flaws in the code developers write (SAST via Snyk Code) and finding known vulnerabilities in the third-party libraries they import (SCA via Snyk Open Source). By integrating directly into the developer workflow, Snyk reduces the cost and time associated with late-stage security patching.

4. Features of the Tool

Snyk offers a robust suite of features tailored for modern development workflows:

- **Comprehensive Scanning:** Combines SAST, SCA, Container scanning, and Infrastructure as Code (IaC) scanning into a single platform.
- **Developer-First CLI:** Allows developers to run rapid, localized tests directly from their terminal without leaving their workflow.
- **Automated Remediation:** Provides actionable fix advice, including the specific package version required to resolve an issue, and can automatically generate pull requests (PRs) to fix vulnerabilities.
- **Vulnerability Database:** Backed by the industry-leading Snyk Intel Vulnerability Database, ensuring high accuracy and low false-positive rates.
- **Continuous Monitoring:** Takes snapshots of project dependencies and monitors them continuously, alerting developers if new vulnerabilities are discovered in existing builds.

5. Installation & Setup

The setup process was conducted locally on a Windows environment targeting a MERN stack project folder named `thermo_aq`. The following steps were executed:

1. **Account Creation:** A standard Snyk account was provisioned via the web portal (snyk.io).
2. **CLI Installation:** The Snyk CLI tool was installed globally via the Node Package Manager using the command: `npm install -g snyk`.
3. **Authentication:** The CLI was authenticated with the Snyk account using the `snyk auth` command, which securely linked the local terminal to the Snyk platform via an OAuth web flow.
4. **Target Navigation:** The terminal was navigated directly into the root directory of the `thermo_aq` project to prepare for localized scanning.

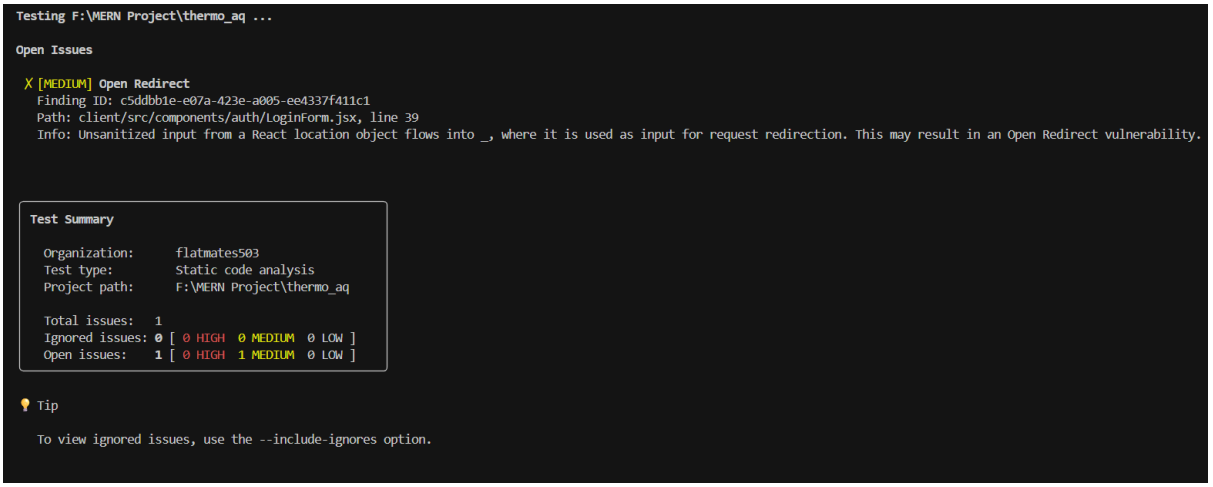
6. Experimentation & Results

The experimentation phase was divided into three distinct execution stages using the CLI, followed by a review on the Web Dashboard.

A. Software Composition Analysis (SCA) The command `snyk test` was executed within the root directory. This engine analyzed the `package.json` and `package-lock.json` files to map the dependency tree and check it against known exploits.

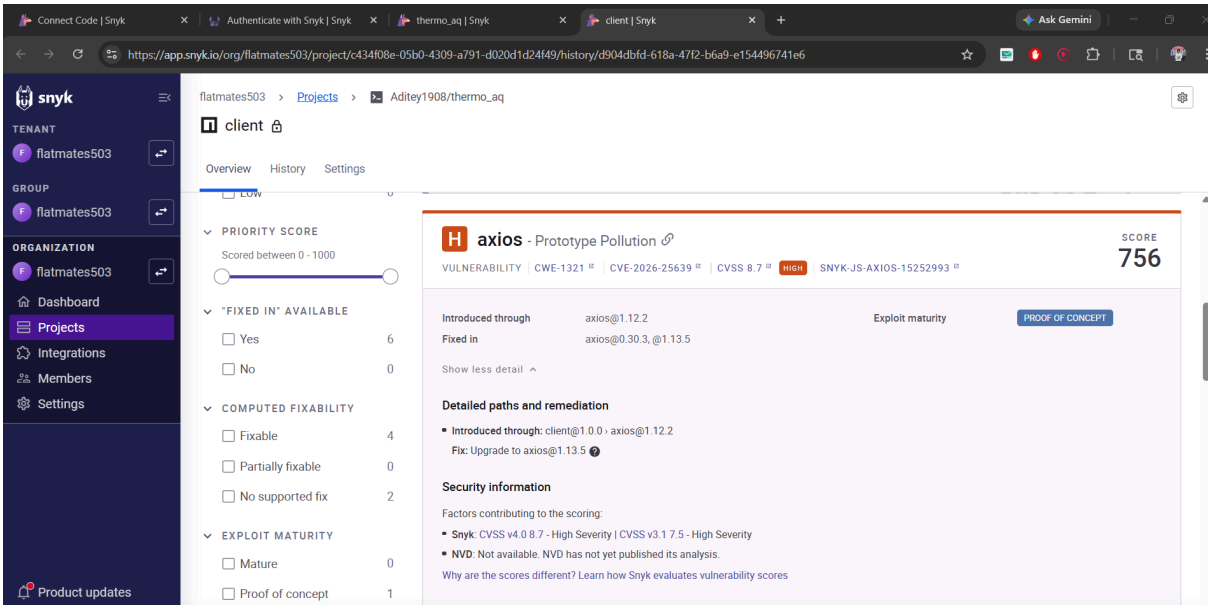
```
F:\MERN Project\thermo_aq>snyk test

Testing F:\MERN Project\thermo_aq...

Organization:      flatmates503
Package manager:   npm
Target file:       package.json
Project name:      thermo_aq
Open source:       no
Project path:      F:\MERN Project\thermo_aq
Licenses:          enabled

✓ Tested F:\MERN Project\thermo_aq for known issues, no vulnerable paths found.

Tip: Detected multiple supported manifests (1), use --all-projects to scan all of them at once.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

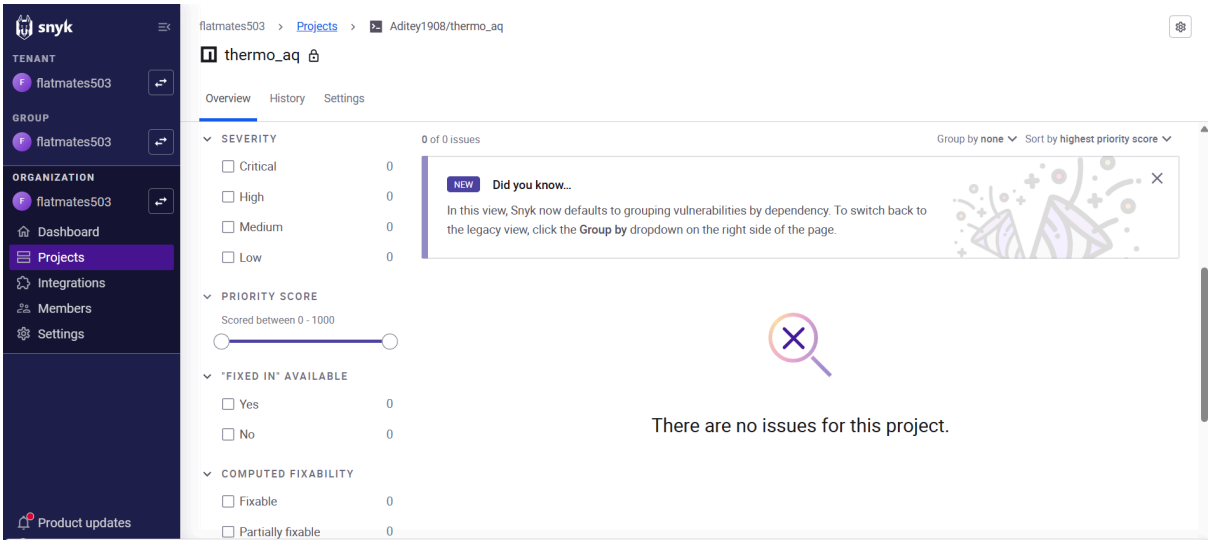
B. Static Application Security Testing (SAST) The command `snyk code test` was executed. This engine analyzed the custom JavaScript logic within the project, scanning for issues such as hardcoded credentials or insecure data handling.



C. Continuous Monitoring & Dashboard Integration To generate a comprehensive visual report, the command `snyk monitor --all-projects` was executed. Snyk successfully identified both the backend (`thermo_aq`) and frontend (`client`) configurations and pushed the snapshots to the Web UI.

```
F:\MERN Project\thermo_aq>snyk monitor --all-projects --org=de459b6c-b466-480f-96dc-c466105751df

Monitoring F:\MERN Project\thermo_aq (thermo_aq)...

Explore this snapshot at https://app.snyk.io/org/flatmates503/project/97ea10e0-81af-4f24-be4d-991d2058557d/history/8a1528bb-a01b-44c0-b693-fb75f0f203a6

Notifications about newly disclosed issues related to these dependencies will be emailed to you.


-------------------------------------------------------

Monitoring F:\MERN Project\thermo_aq (client)...

Explore this snapshot at https://app.snyk.io/org/flatmates503/project/c434f08e-05b0-4309-a791-d020d1d24f49/history/d904dbfd-618a-47f2-b6a9-e154496741e6

Notifications about newly disclosed issues related to these dependencies will be emailed to you.
```





7. Analysis of Outcome

The results generated by Snyk provided immediate, actionable intelligence. Snyk successfully categorized the discovered vulnerabilities into four severity tiers: Critical, High, Medium, and Low.

- **Dependency Assessment:** The SCA scan revealed that several open-source packages in the MERN stack were outdated and carried known CVEs (Common Vulnerabilities and Exposures). Snyk not only identified these but provided the exact upgrade path required (e.g., upgrading a specific package from v1.2 to v1.3).
- **Architecture Recognition:** By running the `--all-projects` flag, Snyk demonstrated its capability to understand complex project structures, correctly separating the risk profile of the Node.js backend from the React frontend client.
- **Remediation Feasibility:** The detailed breakdown provided on the Snyk Dashboard allowed for rapid triage. The "Fix" advice is highly developer-centric, focusing on the minimum required changes to secure the application without breaking existing functionality.

8. Conclusion

The experimentation with Snyk successfully demonstrated the immense value of integrating automated security tools directly into the development environment. By utilizing both the CLI for rapid, iterative testing and the Web Dashboard for comprehensive monitoring, developers can proactively secure their codebases. Snyk effectively bridges the gap between software engineering and cybersecurity, proving that robust application security can be achieved efficiently without compromising development speed.